

Ego-Free Wellbeing Bot

Technical Documentation

1. Project Overview

Name: Ego-Free Wellbeing Bot

Type: Anonymous Telegram bot for emotional self-regulation

Objective: The bot provides simple **coping tools** for 8 specific emotional states. These tools are based on **meta-analyses** of recent academic psychological research related to these issues, combined with **Christian spirituality** expressed through **patristic citations** and academic publications on spirituality.

Additionally, the bot aims to show users that such emotional difficulties may be linked to **ego** / **ego attachment**. It demonstrates a coping approach from a **patristic perspective** – which may be a novel insight for many users.

Finally, the bot generates a **summary** of the suggested tools and some conclusions for the user's future reference.

The system is implemented as a **hybrid decision system** combining a rule-based approach, retrieval-based knowledge base search, and potential integration of AI-based response generation.

Target audience: Anyone experiencing anxiety, sadness, anger, existential frustration, loneliness, shame, overwhelm, or spiritual dryness.

2. Eight Emotional States – Rationale

The 8 emotions were selected based on:

- **Prevalence in contemporary mental health research** (anxiety, sadness, anger, loneliness, shame, overwhelm)
- **Existential and spiritual challenges of modern society** (existential frustration, spiritual dryness) – this adds novelty and addresses a real need.

For each emotion, **Google Labs AI** was queried with:

"What are the most effective and simple tools to cope with [emotion]?"

The search prioritised:

- Peer-reviewed psychological research (CBT, mindfulness, behavioural activation)

- Spiritual traditions (Orthodox patristic texts, contemplative practices)
- Handbooks and meta-analyses

2.1 Source Distribution (over 50 references)

Emotion	Psychology sources	Spiritual sources	Key handbooks / meta-analyses
Anxiety	8	3	CBT meta-analysis, MBCT, Philokalia
Sadness	6	2	BA therapy, Frankl, Psalms
Anger	7	3	Anger management protocols, Evagrius, Cassian
Existential frustration	5	2	Frankl, Existential analysis, Orthodox theology
Loneliness	5	2	Loneliness CBT, MiL study, solitude prayers
Shame	6	2	CFT, Gilbert, shame externalisation
Overwhelm	4	2	GTD, productivity research, simplicity teachings
Spiritual dryness	2	5	Spiritual dryness studies, contemplative endurance, patristic texts
Total	43	21	

Why these sources are optimal for the bot:

- **Simplicity:** each tool is described in one sentence, easy to apply immediately.
- **Duality:** offers both psychological (CBT, mindfulness) and spiritual (prayer, humility, nepsis) paths – respects user choice.
- **Science + tradition:** psychological tools are based on systematic reviews; spiritual tools on 1500+ years of pastoral experience.
- **Keyword-based matching:** keywords stored in CSV allow the bot to select the most relevant tool based on the user's own description.

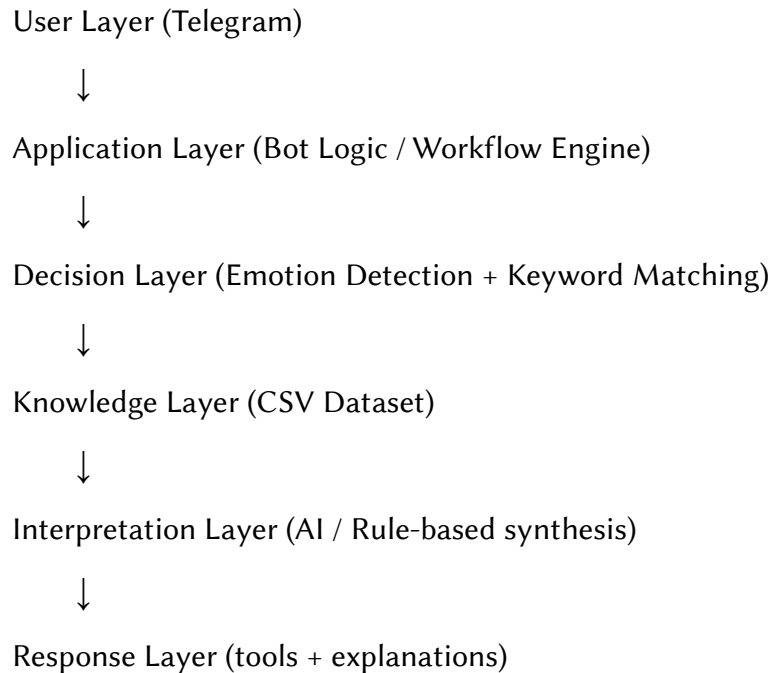
3. System Architecture

The bot is built as a **state machine** that processes user messages via the Telegram API and responds with predefined or CSV-driven answers.

3.1 Architectural Model

The system implements a **multi-layer architecture**:

text



3.2 Technology Stack

Layer	Technology
Programming language	Python 3.11+
Bot framework	python-telegram-bot v20+ (async)
Data processing	pandas (CSV)
Runtime environment	Anaconda environment (or standard Python)
Event loop patch (for Jupyter)	nest_asyncio
Deployment	Docker / Terminal / Anaconda Prompt (24/7) or Jupyter (testing)

3.3 Architectural Principles

The system is built on the following principles:

- **Explainability** – every recommendation is traced to a specific CSV row.
- **Modularity** – CSV, logic, and AI are separated.
- **Deterministic core + optional AI** – base logic works without AI.
- **Extensibility** – new emotions or tools can be added easily.

4. Created Dataset – CSV Knowledge Base

The file `knowledge_base.csv` contains **10 mandatory columns** (the bot will not start if any are missing).

Column	Type	Description	Example
emotion	string	One of 8 emotions	anger

keywords	string	Space-separated words for matching user description	injustice conflict blame
source_type	string	psychology, spiritual, or existential	psychology
tool_name	string	Short name of the tool	cognitive reframing
tool_description	string	One-sentence explanation	Challenge interpretation of events.
citation	string	Source reference	Evidence review
ego_explanation	string	How emotion relates to ego attachment	Anger protects ego identity
youtube_query	string	Search query for YouTube	anger CBT techniques
patristic_explanation	string	Patristic perspective	Anger is misdirected energy
priority	integer	1 = highest relevance	1

4.1 How the Bot Uses the CSV

1. **Filtering** – `data[data["emotion"] == emotion]` retrieves all tools for the detected emotion.
2. **Keyword scoring** – The user's description (step 2) is tokenised; the bot counts how many keywords from each CSV row appear in the description. Higher matches → higher score.
3. **Priority tie-breaker** – When scores are equal, the row with the lower priority number (higher priority) is chosen.
4. **Randomisation** – A small random factor prevents identical outputs for identical inputs.
5. **Source type filtering** – Based on the user's response (psychological, spiritual, or both), the bot keeps only rows with the matching `source_type`.

Thus, the CSV acts as a simplified **knowledge graph**, where each row represents a node linked to an emotion category and a set of key characteristics. This approach aligns with **retrieval-based architectures**.

5. AI Agent

The system implements a **hybrid agent** combining:

- **Rule-based processing** (hard-coded transition logic)
- **Retrieval-based reasoning** (knowledge base search)

- **Template-based generation** (empathy templates)

This approach corresponds to modern **Explainable AI (XAI)** systems, where decisions are formed not directly by a neural network but through a combination of data and logic.

5.1 Step-by-Step AI Decision Process

1. **Step 1** – Bot displays emotion list; user enters number or name.
2. **Step 2** – `normalize_emotion()` converts input to a standard emotion string; stores in state.
3. **Step 3** – User describes their situation. Bot stores description and returns an empathy template (selected by emotion).
4. **Step 4** (merged with step 3) – Bot immediately asks about tool preference.
5. **Step 5** – `find_best_tools()`:
 - Filters CSV by emotion
 - Computes score for each tool based on keyword intersection with user's description
 - Sorts and selects top 2
 - Filters by `source_type` according to preference
 - Formats tool text
6. **Steps 6–8** – Simple yes/no logic for ego explanation, YouTube links, patristic view.
7. **Step 9** – Attempts to call `generate_ai_summary()` (stub); if fails, uses fallback summary.

6. Code Architecture – Technical Description

Structure: Imports → CSV loading → helper functions → state machine → Telegram handler → main block.

6.1 Key Functions

Function	Role
<code>normalize_emotion(text)</code>	Converts user input (e.g., "3", "angry", "rage") to a standard emotion string.
<code>find_best_tools(emotion, user_text)</code>	Returns formatted tool text and top 2 CSV rows.
<code>workflow(user_id, text)</code>	Main state machine. Steps 1-9. Stores <code>user_state</code> dictionary.
<code>handle_message(update, context)</code>	Async wrapper; calls <code>workflow()</code> and sends response.

6.2 User State Dictionary

Each user is identified by `user_id` (Telegram chat ID). The state dictionary stores:

python

```
user_state[user_id] = {  
    "step": 1..9,  
    "emotion": "anxiety",  
    "reason": "user's description",  
    "preference": "both" / "psychological" / "spiritual",  
    "selected_tools": [CSV rows],  
    "tools_text": formatted tool string  
}
```

The bot advances to the next step only after receiving user input. If the user sends `/start`, the state resets to step 1.

7. End-to-End Example

User: `/start`

Bot: Displays emotion list (step 1).

User: 1 (or anxiety)

Bot: Asks: *"In 1–2 sentences, describe why you feel this way."* (step 2)

User: I keep worrying about my job interview tomorrow.

Bot: Empathy: *"It's understandable to feel worried when things are uncertain."* Then immediately: *"Would you like to receive a psychological tool, a spiritual tool, or both?"* (step 3 + merged step 4)

User: both

Bot:

- CSV matches anxiety + keyword worry → computes scores.
- Displays top 2 tools (e.g., cognitive restructuring + breath prayer).
- Then asks: *"Do you want to know how our ego is connected to that?"* (step 5)

User: yes → Bot explains ego attachment, then asks about videos.

User: yes → Bot provides 5 YouTube search links (steps 6–7).

User: yes to patristic view → Bot shows patristic_explanation from CSV (step 8).

User: yes to summary → Bot generates fallback summary (or AI summary if implemented).

Bot: Resets state, invites `/start` again.

8. Dataset Construction Methodology

1. **Emotion selection** – Based on Google Labs API queries for "most common psychological issues" → 8 emotions.
 2. **Tool collection** – For each emotion, searched academic databases (Google Scholar, PubMed) and spiritual reference works. Selected **simple, practically applicable** tools (max 2 sentences).
 3. **Source classification** – Each tool labelled as psychology, spiritual, or existential.
 4. **Keyword extraction** – Manually defined 3–5 keywords per tool that a user might mention.
 5. **Priority assignment** – 1 = most recommended (based on meta-analyses or patristic consensus).
 6. **CSV construction** – Over 50 rows, each row corresponds to one tool. The CSV serves as the **knowledge layer** – can be updated without changing Python code.
-

9. Docker Containerisation

To ensure portability and reproducibility, the system was containerised using Docker. The Docker container includes:

- Python runtime environment
- bot.py (main code)
- knowledge_base.csv (knowledge base)
- requirements.txt (dependencies)

Dockerfile build process:

1. Set base environment (python:3.10-slim)
2. Copy project files
3. Install dependencies
4. Run the bot

Advantages:

- Environment independence (runs on any system)
- Simple deployment
- 24/7 operation capability (server)

- Cloud-deployment ready

Run command:

bash

docker run -d ego-bot

Security: Environment variables are used:

- TELEGRAM_TOKEN
- OPENAI_API_KEY

10. Limitations and Future Improvements

Limitation	Proposed Improvement
No generative AI for summary	Integrate Copilot / GPT API for personalised summaries.
Keyword matching is primitive	Use sentence embeddings (e.g., sentence-transformers) for semantic similarity between user text and tool descriptions.
No conversation memory across sessions	Add a lightweight database (SQLite) to store past interactions.
YouTube links are search URLs, not direct videos	Use YouTube Data API to retrieve actual video IDs and embed them. Add music.
Bot runs only while script is active	Deploy on cloud server (Render, Railway, DigitalOcean) with a process manager (systemd, pm2).

11. Summary

Ego-Free Wellbeing Bot demonstrates the practical application of data science (curating a structured knowledge base, keyword-based retrieval, state machine design) to create an accessible, anonymous emotional support tool. It bridges psychological and spiritual traditions, respects user autonomy (choice of tool type), and is fully transparent – all responses are retrieved from a CSV. The project is ready for deployment and can be extended with generative AI for even more personalised interactions.